

Tut-6

Write an ALP to inspect an array of 12 byte locations and check if the location has an even no. If it has an even no, divide the no by 2 and store the result in the same memory location

```
.model tiny
.data
DAT1 DB 01,02,03,04,05,06,07,08,09,45,56,10
.code
.startup
    MOV SI, OFFSET DAT1
    MOV CL,12
    CLD
l1:  lodsb
    mov ah,al
    ror ah, 1
    jc l2
    shr al,1
    mov [si-1],al
l2:  dec cx
    jnz l1
.exit
end
```

Q1. As a defence security engineer, you have been asked to write a ALP program for 8086 to encrypt war messages so that they are not decoded/intercepted by the enemy.

```
warmessage db "defend attack at Kyiv"
```

```
len db 21
```

```
shifter db 5
```

```
encrypted_message db 21dup(?)
```

The encryption technique assumes the string is stored at **warmessage**, with its length stored in **len**. Note that the encrypted message should be stored in **encrypted_message**. The following is done with the war message:

a. The string in *warmessage* should be reversed and stored in *encrypted_message* (e.g. the *encrypted_message* becomes "viyK ta kcatta dnefed")

b. Then, the bits of each byte in this *encrypted_message* is flipped, rotated right by value stored in the memory location **shifter**, and stored back in the **encrypted_message**.

Write an ALP which does this intended functionality for 8086. Use string operations wherever applicable. You can use only SI, DI, BX, AL and CL register.



```
warmessage db "defend attack at Kyiv"  
len db 21  
shifter db 5  
encrypted_message db 21dup(?)
```

```
lea si, warmessage  
lea di, encrypted_message  
mov bh, 00  
mov bl, len  
add si, bx  
dec si  
x1: std  
    lodsb  
    cld  
    stosb  
    dec bx  
    JNZ x1
```

```
lea si, encrypted_message  
lea di, encrypted_message  
cld; % not compulsory  
mov bl, len  
mov bh, 00h  
x2: mov cl, shifter  
    lodsb  
        not AL  
    ror al, cl  
    stosb  
        dec bx  
    JNZ x2
```